

OpenJDK (<http://openjdk.java.net/>). It's good for you to be aware of this project. As you might have already guessed, it is a GPL-licensed open source implementation of the Java Standard Edition—i.e., the source code of the JDK that you are so familiar with, is also now available for your scrutiny, in case you don't like something in the current JDK.

So, is this a different Java? No—you write the same Java code. You can install OpenJDK from your Linux distribution's package manager (it may come pre-installed with some distributions). See <http://openjdk.java.net/install/> for installation instructions.

Dealing with practicalities

Due to various reasons, deploying Linux lab-wide may not always be possible. In such cases, it's a good idea to have a single Linux machine in the lab, acting as an SSH server; you can install the necessary SSH client software on other operating systems, which will enable connecting to the Linux machine remotely.

This machine should be of a relatively good configuration, depending on how many students will be using it for their coding and compilation—a dual- or quad-core CPU with 4 GB of RAM and a hard disk of at least 320 GB is a good idea.

For Windows, Putty (<http://chiark.greenend.org.uk/~sgtatham/putty/download.html>) is a widely used SSH client. If writing the code on Windows and copying it to the Linux machine to compile and run, you will also need to download the *pscp* program from the site, which lets you copy files from the local machine to the Linux SSH server.

If you need a GUI session from the Linux server to be accessible on the Windows machine (for example, while doing GUI programming) then investigate the OpenNX server (to be installed on the Linux server machine) and the NoMachine NX client for Windows. A machine with the configuration given above should support around 10 user sessions before it starts slowing down. Fine-tuning the desktop manager (use a light one like LXDE or XFCE) and using lighter editors like GVim for writing code, is a good start.

Another option (which does not need a dedicated Linux server machine) is to install Linux in a virtual machine on your desktop. This could also prove useful on a home computer. VirtualBox (<http://www.virtualbox.org/wiki/Downloads>) is virtualisation software that, when installed on your Windows system, will allow you to create a virtual machine, inside which you can install Linux without disrupting your Windows installation.

You will, of course, need some free disk space (8 GB or more) for the virtual machine's disk file. You don't need to burn the Linux installation ISO onto a CD in this case—you can simply instruct VirtualBox to use the ISO image file as a disc inserted in the CD-ROM drive of the virtual machine. This is also a good way to practice installing Linux, and to see how easy it can be.

For Ubuntu, in particular, there is Wubi (<http://wubi-installer.org/>) which lets you install (and uninstall) Ubuntu like any other Windows application, in a simple and safe way, 'with

a single click'. The Ubuntu files are stored in a single folder in your Windows drive, and an option to boot Ubuntu is added to your Windows boot-loader menu. However, hard-disk access is slightly slower than installation to a dedicated partition.

If your Windows drive is very fragmented, the performance will degenerate further. Hibernation is not supported under Wubi. Moreover, the Wubi filesystem is more vulnerable to hard reboots (turning off the power) and power failures than a normal installation to a dedicated partition, which provides a more robust filesystem that can better tolerate such events.

In general, programming on Linux will also require a decent level of familiarity regarding working with shell commands. Get familiar with working with the shell. Try to minimise the use of the mouse :-)

Using your favourite IDE on Linux

If you have been using any IDEs for your development needs, it should be great news that two very popular IDEs—NetBeans and Eclipse—have Linux versions as well, and both of them support C, C++ and Java development. For GNOME-based Linux distributions, Anjuta DevStudio (<http://projects.gnome.org/anjuta/features.html>) is another powerful IDE for C, C++ and Java (and other languages too). All three should be available in your distribution's package manager.

To conclude this article, I would like to urge you to make an honest effort to embrace Linux for programming. It's a much better world to be in. I would love to address any queries/concerns/comments/suggestions that you may have, regarding this article. 

Resources

- Download Ubuntu: <http://www.ubuntu.com/GetUbuntu/download>
- Download Fedora: <http://fedoraproject.org/en/get-fedora>
- 'graphics.ly' like functionality using 'gcc': <http://zaher14.blogspot.com/2007/01/graphics-in-linux.html>
- GNU Compiler Collection: <http://gcc.gnu.org/>
- Brian W. Kernighan and Dennis M. Ritchie, 'The C Programming Language'
- Bjarne Stroustrup, 'The C++ Programming Language'
- Neil Matthew, Richard Stones, 'Beginning Linux Programming'
- StackOverflow.com is a community forum where you can post your programming-related questions. It's language-neutral, which makes it very attractive.
- A basic introduction to SSH: <http://polydistortion.net/doc/ssh.html>
- VirtualBox: www.virtualbox.org, User Manual: www.virtualbox.org/manual/UserManual.html
- www.yolinux.com is a good resource for general Linux information.

By: Amit Saha

The author blogs at <http://amitsaha.wordpress.com> and he welcomes queries and suggestions at amitsaha.in@gmail.com

The author would like to acknowledge Edgar D'Souza, (ED) for 'filling in' bits and pieces (during the editing) which made the article more complete.